

DSA 3050A — Quiz 1

Power Query & Data Modelling using Power BI

Student Name: Selmah Tzindori

Registration Number: 602

Course Code: DSA 3050A

Assessment: Quiz 1

Institution: United States International University

Tool Used: Power BI Desktop

Introduction

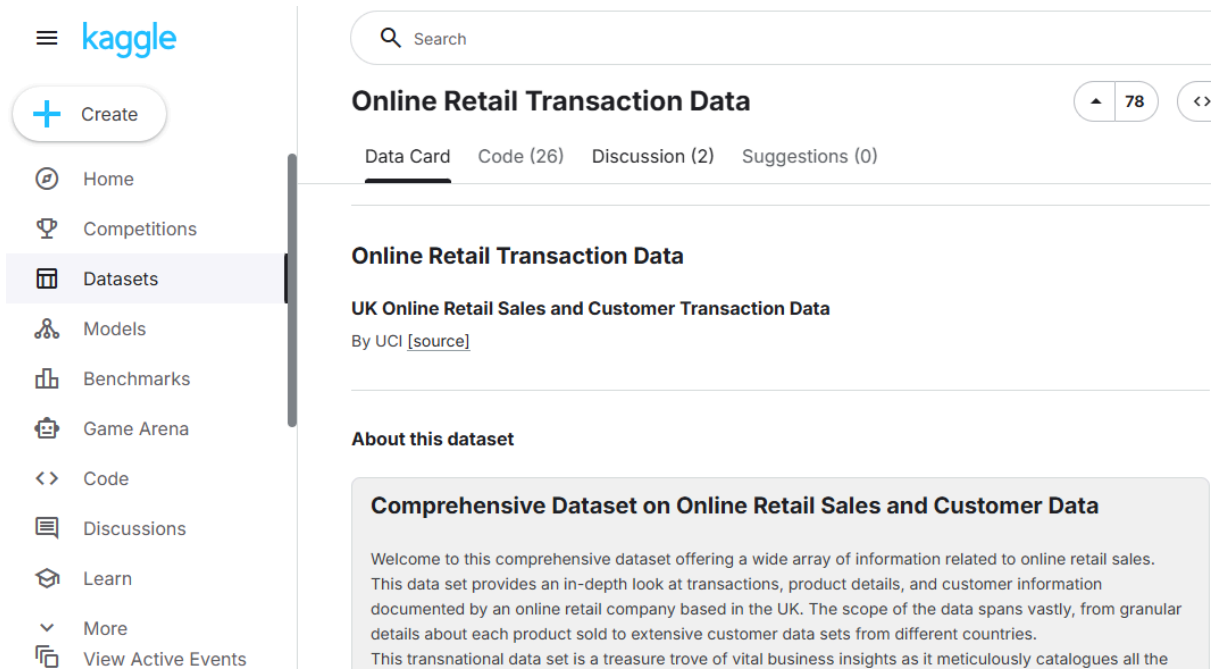
This document presents the submission for **DSA 3050A Quiz 1**, focusing on data transformation using **Power Query** and data modelling in **Power BI**. A real-world retail transactions dataset was used to demonstrate data profiling, cleaning, transformation, and star-schema modelling in accordance with the quiz requirements. All required screenshots, explanations, and evidence are included in this document.

Dataset Used

- **Dataset:** Online Retail Transactions
- **Source:** Kaggle
- **Type:** Real-world business transaction data
- **Main table size:** Over 500,000 rows
- **Tables created:** Fact and dimension tables derived using Power Query

Dataset Source Evidence

Figure 1: Screenshot showing the Kaggle page confirming the real-world source of the Online Retail Transactions dataset.



Minimum Data Requirement Justification

The dataset satisfies the minimum data requirements by containing over 500,000 transaction records, multiple numeric measures (Quantity and UnitPrice), and a date column (InvoiceDate). Related fact and dimension tables were derived from the dataset using Power Query to support a star schema model.

Section A – Power Query

Q1: Data Profiling & Quality Checks

Data Quality Issue 1 – CustomerID

- Column: CustomerID
- Issue: Missing values detected
- Evidence: Column quality showed empty values
- Fix applied: Removed rows with null CustomerID

Figure 2.1: CustomerID column before removing nulls.

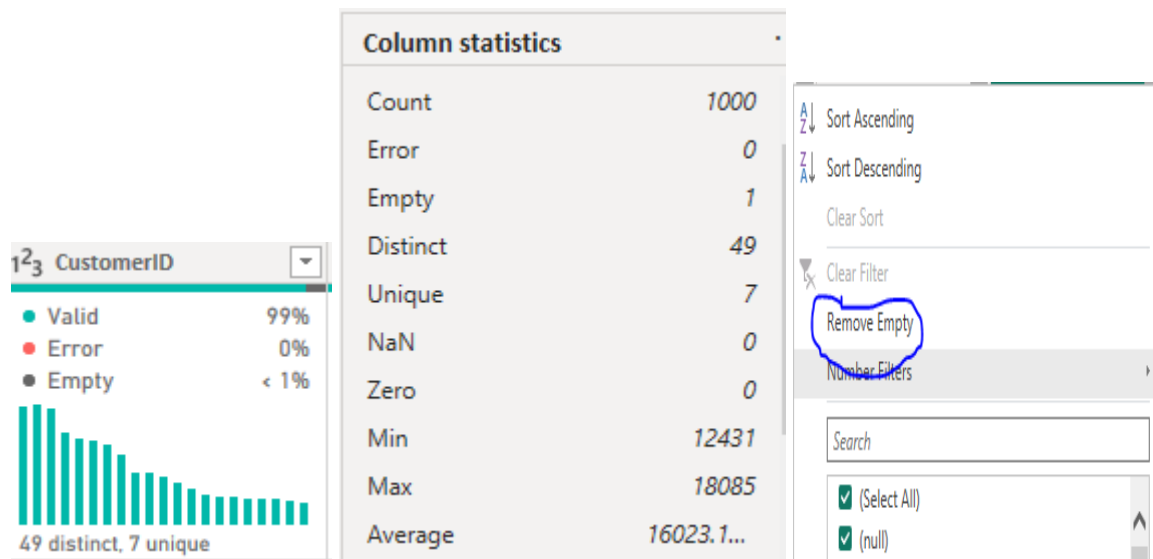
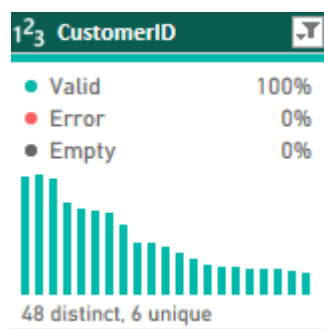


Figure 2.2: CustomerID column after removing nulls. (Click filter icon and deselect null or choose remove empty option)



Data Quality Issue 2 – Quantity

- Column: Quantity
- Issue: Negative values detected
- Evidence: Column distribution highlighted negative quantities
- Fix applied: Filtered Quantity > 0

Figure 3.1: Quantity column before filtering negatives.

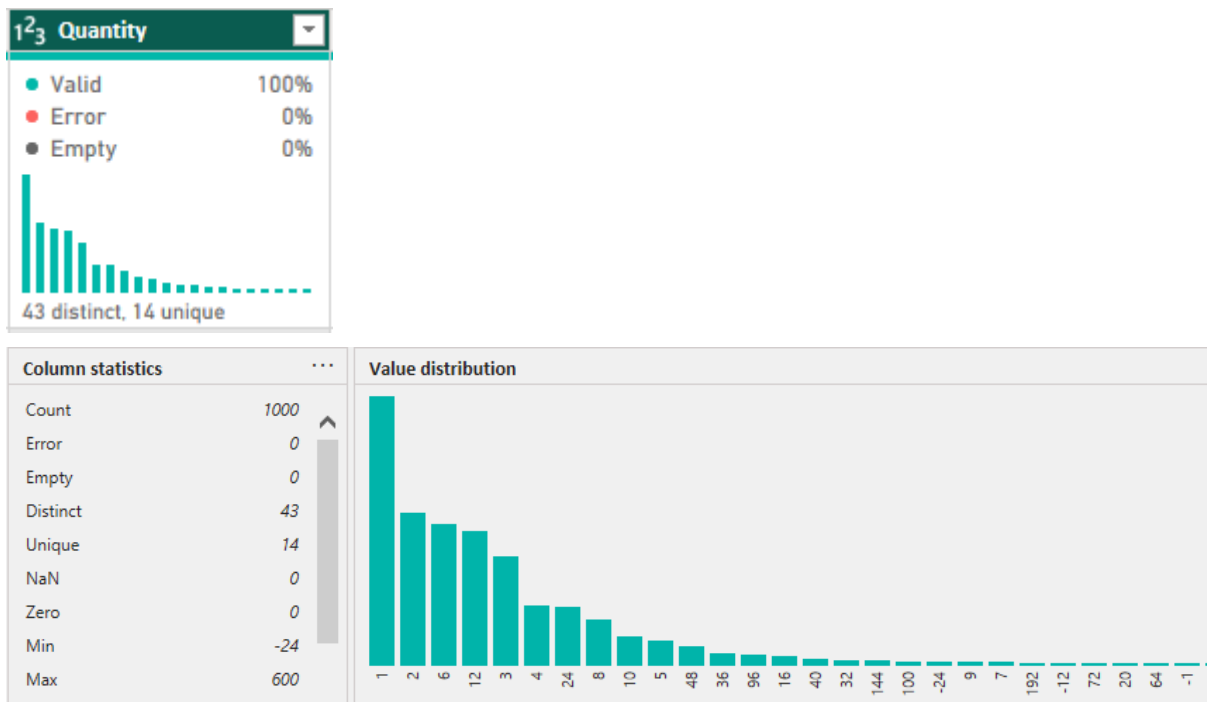


Figure 3.2: Quantity column after filtering negatives(minimum is now one and there are no negative values)

Filter Rows

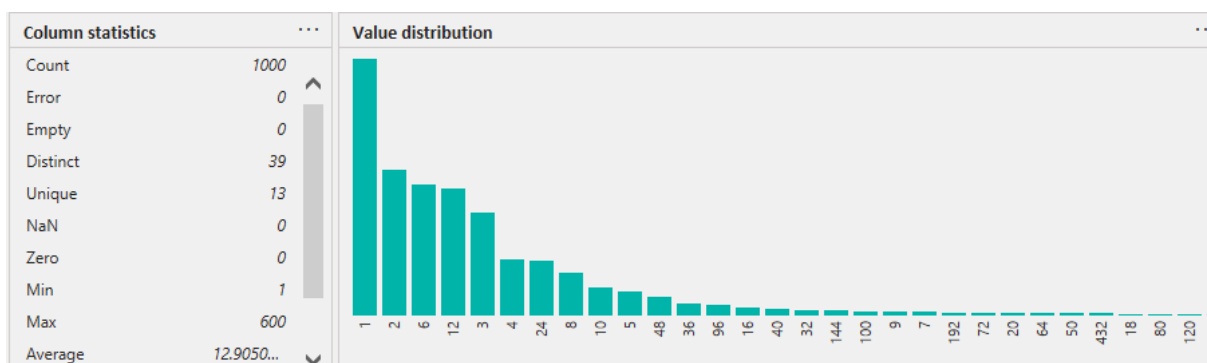
Apply one or more filter conditions to the rows in this table.

☒ Basic ☐ Advanced

Keep rows where 'Quantity'

is greater than

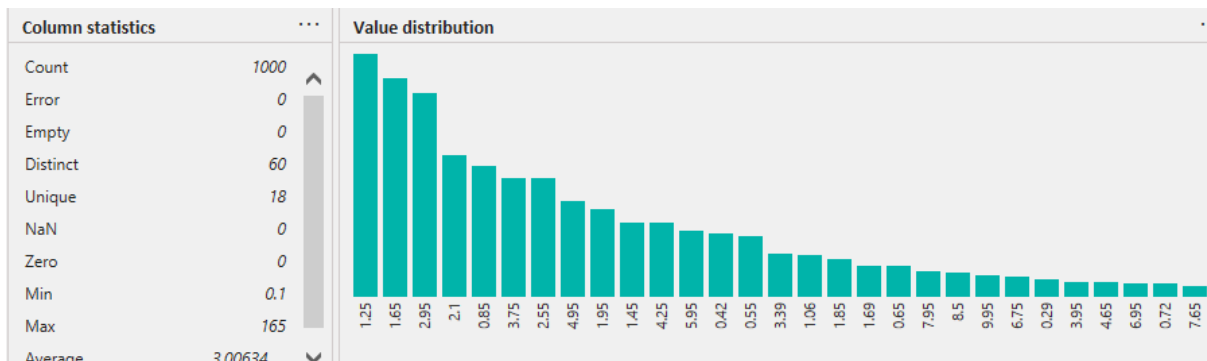
0



Data Quality Issue 3 – UnitPrice

- Column: UnitPrice
- Issue: None (all values positive)
- Evidence: Column distribution shows no negative or zero values
- Fix applied: No action required

Figure 4: UnitPrice column distribution showing all values are positive



Data Quality Issue 4 – Description

- Column: Description
- Issue: Messy text (extra spaces, inconsistent casing)
- Evidence: Preview showed inconsistencies
- Fix applied: Trim + Clean + Proper case

Figure 5.1: Description column before cleaning

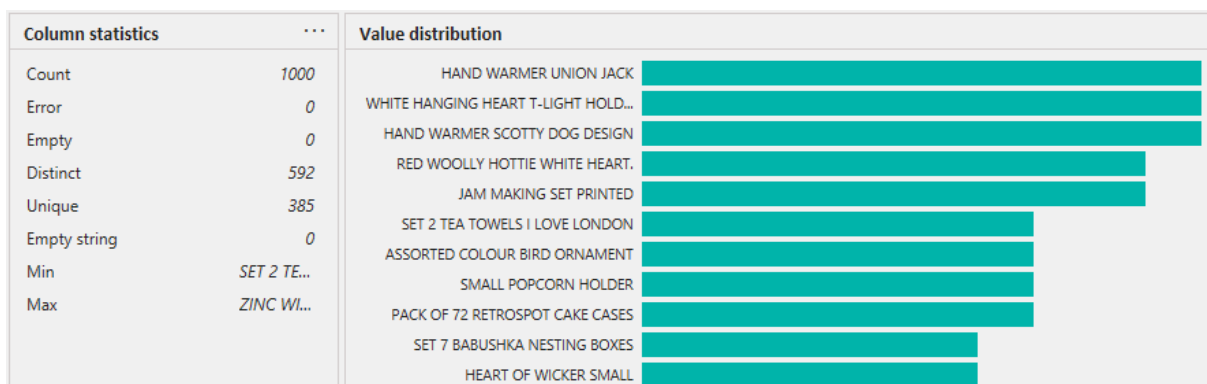
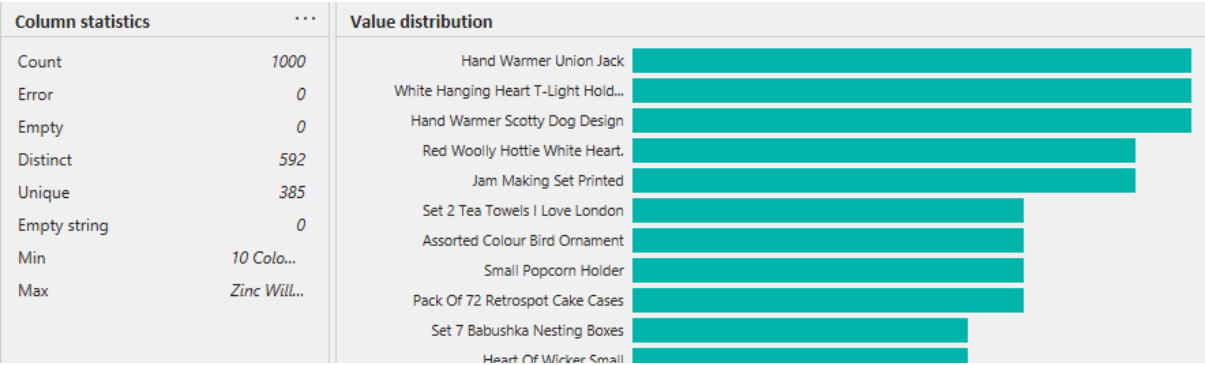


Figure 5.2: Description column after cleaning. (Trim, Clean, lowercase, capitalize each word)



Q2 – Data Types + Locale Conversion

1. InvoiceDate Column

- **Original Issue:** Column was text or datetime incorrectly formatted
- **Action Taken:** Changed data type to **Date/Time**
- **Evidence:** Applied Steps shows “Changed Type”

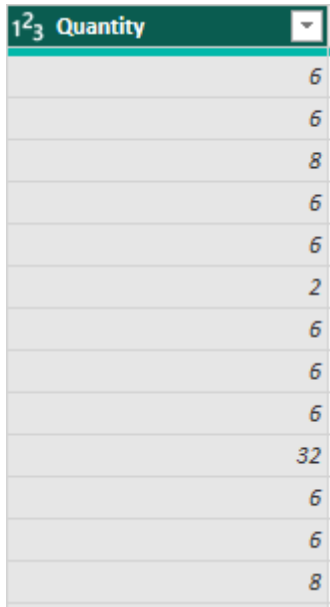
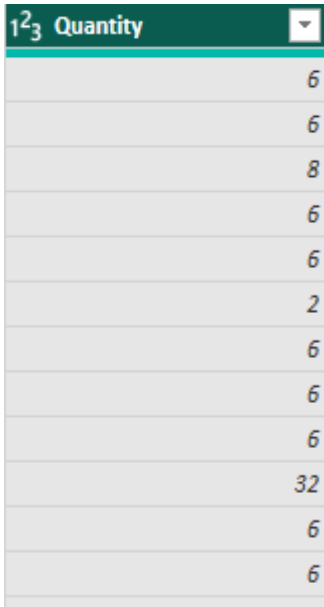
Figure 6: InvoiceDate column before and after data type conversion.

Before	After

2. Quantity Column

- **Original Issue:** Column type was not explicitly set as whole number
- **Action Taken:** Changed data type to **Whole Number**
- **Evidence:** Applied Steps shows “Changed Type”

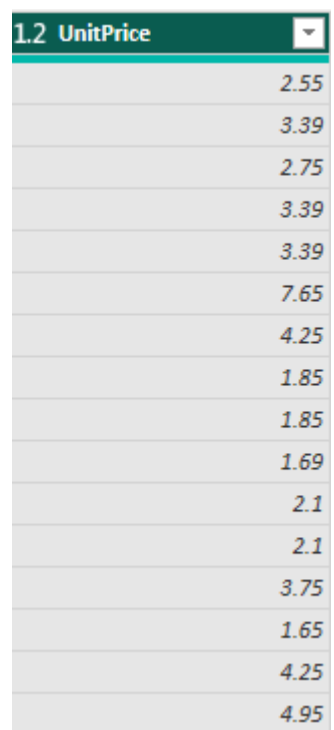
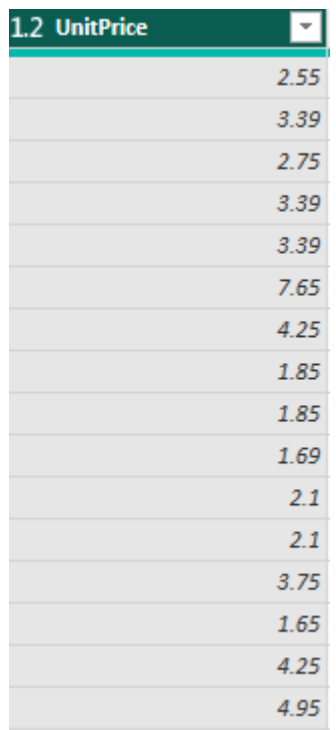
Figure 7: Quantity column before and after type change.

Before	After
	

3. UnitPrice Column

- **Original Issue:** Column may have been formatted incorrectly for locale
- **Action Taken:** Changed type to **Decimal Number** and applied **Locale Conversion**
 - Locale used: **English (United Kingdom)**
- **Evidence:** Applied Steps shows “Changed Type with Locale”

Figure 8: UnitPrice column before and after type conversion using locale.

<i>Before</i>	<i>After</i>
	

4.CustomerID Column

- **Original Issue:** Column type is whole number
- **Action Taken:** Changed type to **Text** to preserve identifiers
- **Evidence:** Applied Steps pane shows “Changed Type”

Figure 9.1: CustomerID column before changing type to Text.

	Quantity	InvoiceDate	UnitPrice	CustomerID
t Holder	6	12/1/2010	2.55	17850
	6	12/1/2010	3.39	17850
anger	8	12/1/2010	2.75	17850
er Bottle	6	12/1/2010	3.39	17850

Figure 9.2: CustomerID column after changing type to Text.

	Quantity	InvoiceDate	UnitPrice	CustomerID
t Holder	6	12/1/2010	2.55	17850
	6	12/1/2010	3.39	17850
anger	8	12/1/2010	2.75	17850
er Bottle	6	12/1/2010	3.39	17850
part.	6	12/1/2010	3.39	17850
es	2	12/1/2010	7.65	17850
older	6	12/1/2010	4.25	17850

Q3– Text Standardization

Description Column

- **Original Issue:**
 - Extra spaces at the start/end of text
 - Non-printable characters (tabs, line breaks)
 - Inconsistent casing (all uppercase, lowercase, mixed)
- **Action Taken:**
 - Applied **Trim** → removed leading/trailing spaces
 - Applied **Clean** → removed non-printable characters

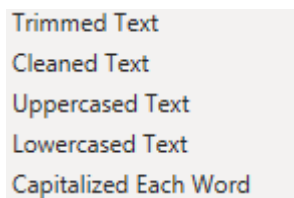
- Applied **Proper Case** → standardized casing (capitalized first letter of each word)

Justification for Text Standardization and Column Naming

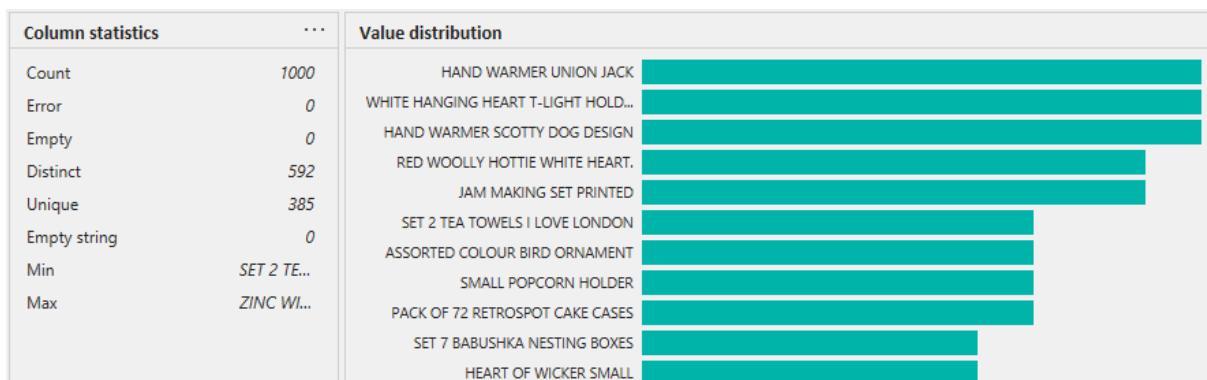
- The Description column was cleaned using **Trim + Clean + Proper Case** to remove extra spaces, non-printable characters, and inconsistent casing, ensuring the text is standardized for analysis. No column renaming was necessary because the existing column names were already clear and descriptive. For example, **Description** accurately represents product descriptions, and other columns such as **CustomerID** and **UnitPrice** are self-explanatory. Renaming would not have added any clarity or value to the dataset.

- **Evidence:**

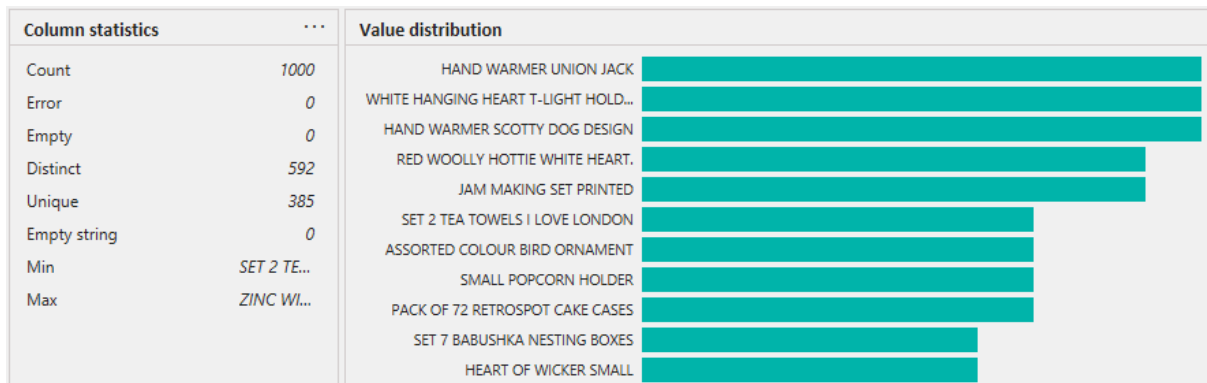
- Applied Steps pane shows **Trim → Clean → Format → Proper Case**



See Figure 5.1: Description column before cleaning



See Figure 5.2: Description column after cleaning



Q4 – Conditional + Custom Columns

Conditional Column – QuantityCategory

Action Taken: Categorized Quantity into Low / Medium / High based on observed range (1–600)

- Ranges were chosen based on the minimum (1) and maximum (600) Quantity values in the dataset.

Rules Applied:

- Quantity > 150 → High
- Quantity > or = 51 → Medium
- Quantity ≤ 50 → Low

Evidence: Applied Steps pane visible

Figure 10.1: QuantityCategory conditional formula bar

Add Conditional Column

Add a conditional column that is computed from the other columns or values.

New column name
QuantityCategory

	Column Name	Operator	Value ①		Output ①
If	Quantity	is greater than	ABC 123 150	Then	ABC 123 High
Else If	Quantity	is greater than or...	ABC 123 51	Then	ABC 123 Medium
Else If	Quantity	is less than or equ...	ABC 123 50	Then	ABC 123 Low

Add Clause

Else ①
ABC 123

Figure 10.1: QuantityCategory conditional results

ABC 123	QuantityCategory
	Low
	Low
	Low
	Low
	Low
	Low
	Low
	Low
	Low
	Low
	Low

Custom Column – SalesAmount

- **Action Taken:** Created a custom column calculating total sales for each invoice row
- **Formula Used:** = [Quantity] * [UnitPrice]
- **Evidence:** Applied Steps pane visible, formula bar shown

Figure 11.1: SalesAmount custom formula bar

Custom Column

Add a column that is computed from the other columns.

New column name

SalesAmount

Custom column formula ①

= [UnitPrice]*[Quantity]

Available columns

index
InvoiceNo
StockCode
Description
Quantity
InvoiceDate
UnitPrice

<< Insert

Figure 11.1: SalesAmount results

ABC 123	SalesAmount
	15.3
	20.34
	22
	20.34
	20.34
	15.3
	25.5
	11.1
	11.1
	54.08
	12.6
	12.6
	30
	9.9
	25.5
	14.85

Q5 – Grouping

- **Action Taken:** Grouped data by **Country**
- **Aggregations Produced:**
 1. TotalQuantity (sum of Quantity)

2. TotalSales (sum of SalesAmount)

- **Evidence:** Group by pane visible

Figure 12.1: Group by aggregation pane

Group By

Specify the columns to group by and one or more outputs.

☐ Basic ☒ Advanced

Country ▾

Add grouping

New column name	Operation	Column
TotalQuantity	Sum ▾	Quantity ▾
TotalSales	Sum ▾	SalesAmount ▾

Add aggregation

Figure 12.1: Group by pane results

A ^B _C Country ▾	1.2 TotalQuantity ▾	1.2 TotalSales ▾
United Kingdom	4269472	7308391.554
France	111472	209024.05
Australia	84209	138521.31
Netherlands	200937	285446.34
Germany	119263	228867.14
Norway	19338	36165.44
EIRE	140525	265545.9
Switzerland	30083	56443.95
Spain	27951	61577.11
Poland	3684	7334.65
Portugal	16122	33439.89
Italy	8112	17483.24
Belgium	23237	41196.34

Section B – Data Modelling

Q6 – Star Schema Identification

- **Fact Table:** Invoices
 - Justification: Contains numeric measures (Quantity, UnitPrice, SalesAmount) that we want to analyze
- **Dimension Tables:**
 - Customer: attributes like CustomerID, Country
 - Product: attributes like StockCode, Description
- **Bridge Table:** Not required because there is no many-to-many relationship

Q7 – Relationships

1. Create Dimension Tables

DimCustomer:

- Columns: CustomerID, Country
- Remove duplicates → **Home** → **Remove Rows** → **Remove Duplicates**

Figure 13.1: Dimension Customer Created

	A _C ^B CustomerID	A _C ^B Country
1	17850	United Kingdom
2	13047	United Kingdom
3	12583	France
4	13748	United Kingdom
5	15100	United Kingdom
6	15291	United Kingdom
7	14688	United Kingdom
8	17809	United Kingdom
9	15311	United Kingdom
10	16098	United Kingdom
11	18074	United Kingdom

DimProduct:

- Columns: StockCode, Description

- Remove duplicates

Figure 12.2: Dimension Product Created

	A ^B _C StockCode	A ^B _C Description
1	85123A	White Hanging Heart T-Light Holder
2	71053	White Metal Lantern
3	84406B	Cream Cupid Hearts Coat Hanger
4	84029G	Knitted Union Flag Hot Water Bottle
5	84029E	Red Woolly Hottie White Heart.
6	22752	Set 7 Babushka Nesting Boxes
7	21730	Glass Star Frosted T-Light Holder
8	22633	Hand Warmer Union Jack
9	22632	Hand Warmer Red Polka Dot
10	84879	Assorted Colour Bird Ornament
11	22745	Poppy'S Playhouse Bedroom

2.Create Relationships

Action Taken: Connected Fact table to Dimension tables

Relationships Created:

1. Fact[CustomerID] → DimCustomer[CustomerID] (Many-to-One, Single, Active)

← New relationship

From table
Dim_Customer

Country	CustomerID
United Kingd...	17850
United Kingd...	13047
United Kingd...	13748

To table
Fact_Sales

Country	CustomerID	Description	index	InvoiceDate	InvoiceNo	Quantity
United Kingd...	17841	3D Christmas ...	2040	Wednesday, ...	536557	1
United Kingd...	17841	Birthday Party...	2045	Wednesday, ...	536557	1
United Kingd...	17841	White Spot R...	2051	Wednesday, ...	536557	1

Cardinality
One to many (1:*)

Cross-filter direction
Single

☒ Make this relationship active
☐ Apply security filter in both directions
☐ Assume referential integrity

2. Fact[ProductCode] → DimProduct[ProductCode] (Many-to-One, Single, Active)

←

New relationship

×

Select tables and columns that are related.

From table

Dim_Product

Description	StockCode
White Hangin...	85123A
White Metal L...	71053
Cream Cupid ...	84406B

To table

Fact_Sales

	index	InvoiceDate	InvoiceNo	Quantity	QuantityCate...	SalesAmount	StockC
...	2040	Wednesday, ...	536557	1	Low	1.25	2273
/...	2045	Wednesday, ...	536557	1	Low	1.25	2295
...	2051	Wednesday, ...	536557	1	Low	1.25	2167

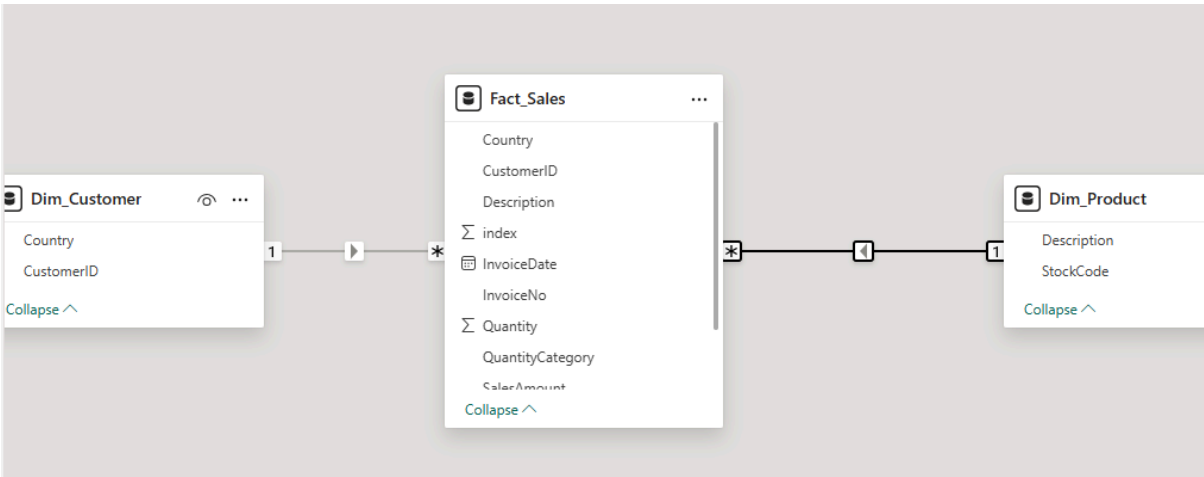
Cardinality

One to many (1:*)

Cross-filter direction

Single

Figure 12.2: Power BI model view with relationships clearly visible



Relationship 1: FactSales → DimCustomer

Property	Value
----------	-------

Cardinality	Many-to-One (Fact → Dimension)
Cross filter direction	Single
Active/Inactive	Active
Why correct	Each CustomerID in FactInvoices may appear many times (many transactions), but each CustomerID in DimCustomer is unique. Single-direction filtering ensures totals and aggregations work correctly from Dimension → Fact.

Relationship 2: FactSales → DimProduct

Property	Value
Cardinality	Many-to-One (Fact → Dimension)
Cross filter direction	Single
Active/Inactive	Active
Why correct	Each ProductCode in FactInvoices may appear many times, but each ProductCode in DimProduct is unique. Single-direction filtering ensures proper aggregation of sales by product.

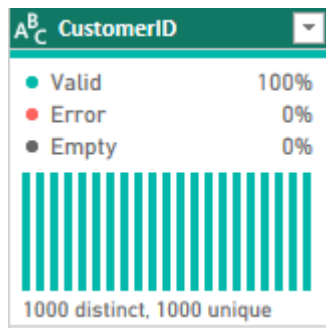
Common Modelling Mistake

- **Example:** Linking **FactSales** → **FactSales** instead of Fact → Dimension, or linking **non-unique keys**
- **Impact:** Totals and aggregations will be **incorrect** because filters won't propagate properly. For example:
 - Sum of SalesAmount by Customer will double-count or fail
 - Visuals may show **wrong totals** or inconsistent values

Q8 – Key Uniqueness

- **Dimension Table Checked:** DimCustomer
- **Key Column:** CustomerID
- **Check Method:** Compared total rows vs distinct values (or Column Statistics in Power Query)
- **Result:** Key is unique → no duplicates found; unique=distinct

Figure 13: Key uniqueness verification for CustomerID (If there were duplicates we would highlight the column and select remove duplicates)



Q9 – Date Table Decision

Should a date table exist?

Answer: Yes

Reasons

1. Ensures proper **time intelligence calculations** (e.g., Year-to-Date, Month-to-Date totals)
2. Allows **consistent filtering and grouping** across multiple tables

Suggested Columns

1. Date
2. Year
3. Month
4. Quarter
5. Week number

Small Data Dictionary

This data dictionary describes each table and column in the model, including keys, measures, and descriptive attributes, to clarify the dataset for analysis.

Table	Column	Meaning	Type	Key/Attribute
FactSales	InvoiceNo	Unique invoice number	Text	Key
FactSales	Quantity	Number of items sold	Whole	Measure
FactSales	UnitPrice	Price per item	Decimal	Measure
FactSales	SalesAmount	Total sales per row (Quantity × UnitPrice)	Decimal	Measure
DimCustomer	CustomerID	Unique customer identifier	Text	Key
DimCustomer	Country	Country of customer	Text	Attribute
DimProduct	ProductCode	Unique product identifier	Text	Key
DimProduct	Description	Product description	Text	Attribute